

# Rapport de Projet NoSQL

## Un Moteur d'Évaluation de Requêtes RDF en Étoile

Franck REVEILLE, Etienne MOUSSA

29 novembre 2025

### Résumé

Ce rapport présente la conception et l'implémentation d'un système de gestion de bases de données RDF en mémoire. Le projet met en œuvre et compare deux approches de stockage : une approche naïve (Giant Table) et une approche indexée (Hexastore). L'accent est mis sur l'évaluation performante de requêtes en étoile (Star Queries) via des heuristiques de sélectivité, ainsi que sur la validation rigoureuse de la correction et de la complétude du moteur par comparaison avec le système de référence InteGraal.

## Table des matières

|          |                                                                |          |
|----------|----------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                            | <b>2</b> |
| <b>2</b> | <b>Architecture de Stockage</b>                                | <b>2</b> |
| 2.1      | Approche Naïve : Giant Table . . . . .                         | 2        |
| 2.2      | Approche Optimisée : Hexastore . . . . .                       | 2        |
| <b>3</b> | <b>Moteur de Requêtes et Optimisations</b>                     | <b>2</b> |
| 3.1      | Traitement des Requêtes en Étoile (Star Queries) . . . . .     | 2        |
| 3.2      | Optimisation par Sélectivité (Heuristique "HowMany") . . . . . | 2        |
| <b>4</b> | <b>Validation : Correction et Complétude</b>                   | <b>3</b> |
| 4.1      | Méthodologie de Comparaison . . . . .                          | 3        |
| 4.2      | Résultats . . . . .                                            | 3        |
| <b>5</b> | <b>Conclusion</b>                                              | <b>3</b> |

# 1 Introduction

Dans le cadre du cours de NoSQL, nous avons développé un moteur capable de charger, stocker et interroger des données RDF. L'objectif principal était de traiter efficacement des requêtes SPARQL de forme "étoile", où plusieurs triplets partagent un même sujet. Pour ce faire, nous avons implémenté une architecture modulaire permettant de comparer les performances d'un stockage séquentiel simple face à une structure d'indexation complexe : l'Hexastore.

## 2 Architecture de Stockage

Le système repose sur l'interface `RDFStorage`, qui définit le contrat pour l'ajout de données et la recherche de motifs.

### 2.1 Approche Naïve : Giant Table

Nous avons d'abord implémenté une solution de référence, la *Giant Table*.

- **Structure** : Elle stocke simplement la liste des triplets dans une collection linéaire (ex : `ArrayList`).
- **Performance** : La recherche d'un motif (`match`) nécessite le parcours intégral de la table, entraînant une complexité linéaire  $O(N)$ . Cette approche sert de base ("baseline") pour mesurer les gains de performance des index.

### 2.2 Approche Optimisée : Hexastore

Pour répondre aux exigences de performance sur des jeux de données volumineux (comme `100K.nt` ou `500K.nt`), nous avons implémenté la structure `RDFHexaStore`.

- **Principe** : L'Hexastore maintient six index correspondant à toutes les permutations possibles des éléments d'un triplet (Sujet S, Prédicat P, Objet O) : SPO, SOP, PSO, POS, OSP, OPS.
- **Implémentation** : Chaque index est une structure de *Map* imbriquée. Par exemple, l'index SPO est représenté par `Map<S, Map<P, List<O>>>`.
- **Avantage** : Cette redondance permet de récupérer les résultats de n'importe quel motif de triplet atomique avec une complexité proche de  $O(1)$  (temps constant), au prix d'une consommation mémoire accrue (x6).
- **Dictionnaire** : Pour optimiser la mémoire et les comparaisons, toutes les chaînes de caractères (URI, Littéraux) sont encodées en entiers via un dictionnaire bidirectionnel.

## 3 Moteur de Requêtes et Optimisations

### 3.1 Traitement des Requêtes en Étoile (Star Queries)

Une *StarQuery* est définie par une variable centrale commune à une liste de motifs de triplets. Notre stratégie d'évaluation repose sur l'algorithme de jointure par intersection : 1. Pour chaque motif de la requête, on interroge le `RDFStorage` pour obtenir les candidats possibles pour la variable centrale. 2. On effectue l'intersection de ces ensembles de candidats. 3. Les entités restantes sont celles qui satisfont toutes les conditions.

### 3.2 Optimisation par Sélectivité (Heuristique "HowMany")

L'ordre dans lequel les intersections sont réalisées impacte massivement les performances.

- **Problème** : Commencer par un motif peu sélectif (ex : `?x rdf:type foaf:Person`) génère des milliers de candidats intermédiaires inutiles.

- **Solution** : Nous avons implémenté une méthode qui interroge l'Hexastore pour connaître la taille du résultat (*count*) sans matérialiser les données.
- **Algorithme** : Avant d'exécuter la requête, le moteur trie les atomes du plus sélectif (celui qui a le moins de résultats) au moins sélectif. Cela réduit drastiquement la taille des ensembles manipulés lors des intersections successives.

## 4 Validation : Correction et Complétude

La rapidité d'un moteur est inutile si les résultats sont erronés. Nous avons mis en place un protocole de test strict utilisant la bibliothèque **InteGraal** comme Oracle.

### 4.1 Méthodologie de Comparaison

La classe de vérification suit les étapes suivantes :

1. **Chargement Miroir** : Le même fichier de données (ex : `sample_data.nt`) est chargé simultanément dans notre `RDFHexaStore` et dans le `SimpleInMemoryGraphStore` d'InteGraal.
2. **Exécution Parallèle** : Chaque requête `StarQuery` est convertie en requête formelle (via `asFOQuery()`) pour InteGraal, et exécutée en parallèle sur notre moteur.
3. **Comparaison Ensembliste** : Les résultats (valeurs de la variable centrale) sont stockés dans deux ensembles (`Set`). Nous vérifions l'égalité stricte  $Set_{Hexastore} \equiv Set_{Oracle}$ .

### 4.2 Résultats

Cette procédure garantit :

- La **Correction** : Nous ne produisons pas de résultats que l'Oracle ne trouve pas.
- La **Complétude** : Nous trouvons tous les résultats que l'Oracle trouve.

Les tests unitaires fournis (`RDFHexaStoreTest`, `StarQueryTest`) valident par ailleurs la bonne construction des structures de données et le parsing des fichiers.

## 5 Conclusion

Ce projet a permis de mettre en évidence l'importance des structures d'indexation dans les bases de données NoSQL orientées graphe. L'implémentation de l'Hexastore, couplée à l'optimisation de l'ordre des jointures, offre des performances nettement supérieures à l'approche naïve. La validation croisée avec InteGraal assure la fiabilité du système, fournissant une base solide pour de futures extensions (persistance disque, requêtes plus complexes).